WEB DEVELOPMENT PROJECT REPORT

Image Slider Library

A responsive, accessible carousel built from browser fundamentals

PROJECT SCOPE

DOM structure / responsive CSS / modular JavaScript / infinite motion

Prepared for the Web_development_mit repository

12 October 2026

Introduction

An image slider combines content layout, input handling, and visual state into a compact interface. This project builds that pattern from standard HTML, CSS, and JavaScript so its behavior is visible, adjustable, and easy to reuse without a third-party carousel package.

The demo presents five landscape photographs in a single responsive viewing window. Visitors can move one image at a time with previous and next buttons, select a specific image from generated indicator dots, or use the left and right arrow keys while the slider is focused. The active image, counter, and accessible status message stay in step with the track.

The implementation is organized around three files. index.html supplies semantic slide content and data attributes, style.css defines the viewport and visual states, and slider.js exports an ImageSlider class that creates its navigation controls and manages transitions. A README explains how to run and adapt the component, and image-credits.md lists the public sample image sources.

Design goals

- Make the horizontal movement continuous and directionally consistent when the user crosses either end of the slide list.
- Keep the wrapper fluid, crop photographs without stretching them, and adjust the viewing window at narrow breakpoints.
- Expose clear control names and current-slide feedback, support keyboard input, and respect reduced-motion preferences.
- Keep the demo dependency-free at runtime and publish the component in a self-contained project directory.

The visual demo uses a quiet editorial layout to keep attention on the photographs and controls. The library remains independent of that styling: host pages can reuse the data-attribute structure with a different design system.

Implementation details

Document structure and component contract

The slider root is identified by data-image-slider. Inside it, data-slider-viewport marks the clipped viewing window and data-slider-track identifies the flex row that holds the figures. Each supplied figure carries data-slider-slide. Previous and next buttons, the generated-dot container, and optional counter and live-status nodes use their own data attributes, keeping JavaScript selectors separate from presentation classes.

Each figure contains an image with descriptive alternative text and can include a caption. The demo starts with five figures in its HTML; JavaScript reads that list to generate exactly five dot buttons. A host page can add or remove figures and the dot count follows the supplied content on initialization.

Track layout and image sizing

The viewport hides overflow while the track lays out slides in a horizontal flex row. Every slide occupies one viewport width. The component calculates the translation as the negative of the visual slide position multiplied by the current viewport width, then applies that pixel value with translateX(). The CSS transition interpolates between positions to create the requested slide motion. The translateX() function repositions an element along the horizontal axis (MDN Contributors, 2026b).

The image element fills its slide box with object-fit: cover, preserving the source ratio and cropping any excess edge area when the frame has a different ratio. This avoids stretched photographs while keeping the viewport fully covered (MDN Contributors, 2026a).

The desktop viewing window uses a wide aspect ratio. Media queries move to a taller crop on smaller screens, reduce side spacing, and preserve adequate space for arrows and captions. The ResizeObserver recalculates the track offset when the component width changes, so the same logical slide remains aligned after resizing.

Interaction architecture

Controls and state

ImageSlider stores the active logical index separately from the track position. Arrow and keyboard inputs enqueue one-step commands. A dot click targets a specific index and replaces any commands waiting behind the active animation, so the user can redirect to a chosen image without leaving the track in an invalid position. Each transition updates the active dot, visible counter, slide accessibility state, and live announcement.

When a change begins, the component emits a bubbling slidechange CustomEvent with the new index, slide element, and input source. A host page can listen for this event to synchronize captions, analytics, or another small interface without modifying the slider class.

Seamless boundary wrapping

For a list of n slides, the component inserts a clone of the final slide before the first slide and a clone of the first slide after the final slide. The real slides therefore occupy visual positions 1 through n. Moving forward from the last real slide animates to the final clone at position n + 1; once the transition ends, the track jumps without animation to position 1. Moving backward from the first real slide uses position 0 and then resets to position n. The duplicate content makes both jumps visually indistinguishable from uninterrupted movement.

The transitionend event completes each move and a timeout provides a fallback if a browser does not dispatch the expected event. The queue prevents rapid arrow input from sending the track beyond the clone boundaries. A one-slide slider does not create clones or accept navigation commands.

Accessibility and motion

The arrow buttons have explicit accessible names, every generated dot names its destination, and aria-current identifies the selected dot. Only the selected original slide is exposed to assistive technology; the boundary clones are hidden from the accessibility tree. Arrow keys work while focus is within the slider, and the focus ring remains visible.

A prefers-reduced-motion media query reduces transition time for visitors who ask for less motion. Image alt text describes the landscape content, while captions add place and scene labels without repeating the accessible control names.

Results and verification

The project files are present and the JavaScript module passes Node syntax parsing. A structural check finds five supplied slide figures and five image elements, plus the dynamic dot builder, loop-slide cloning logic, translateX positioning, overflow clipping, and responsive image rules. These checks confirm the component structure, but they do not verify the rendered browser interaction.

The source excerpt below shows the command queue and the two boundary destinations. A live application screenshot and browser interaction review still need an accessible browser session; the current execution environment cannot reach its local preview server. This is recorded explicitly here rather than presenting a mock image as an application capture.

Figure 1. Slider boundary-navigation source excerpt

slider.js — LOOP NAVIGATION

```
144   processQueue() {
145     if (this.isAnimating || this.commandQueue.length === 0 || this.destroyed) {
146       return;
147     }
148
149     const command = this.commandQueue.shift();
150     const targetIndex =
151       command.type === "step"
152         ? (this.currentIndex + command.direction + this.slideCount) % this.slideCount
153         : command.index;
154
155     if (targetIndex === this.currentIndex) {
156       this.processQueue();
157       return;
158     }
159
160     let targetVisualIndex = targetIndex + 1;
161     const movingForward =
162       command.type === "step"
163         ? command.direction > 0
164         : this.currentIndex === this.slideCount - 1 && targetIndex === 0;
165     const movingBackward =
166       command.type === "step"
167         ? command.direction < 0
168         : this.currentIndex === 0 && targetIndex === this.slideCount - 1;
169
170     if (movingForward && this.currentIndex === this.slideCount - 1 && targetIndex === 0) {
171       targetVisualIndex = this.slideCount + 1;
172     } else if (movingBackward && this.currentIndex === 0 && targetIndex === this.slideCount - 1) {
173       targetVisualIndex = 0;
174     }
175
176     this.animateTo(targetIndex, targetVisualIndex, command.source);
177   }
```

APPLICATION OUTPUT SCREENSHOT

Pending capture from a browser that can access the local demo.

Conclusion

The image slider library meets the core structural and interaction requirements in its source: it builds one dot per slide, moves a flex track with horizontal transforms, wraps across both boundaries using edge clones, and recalculates its position as the viewport changes. Its class boundary, data-attribute markup contract, and slidechange event make it practical to adapt to other page content.

The main remaining verification step is a live browser review of the application output. The source currently depends on public image and font URLs, so the full visual demo also requires an internet connection. The code is organized so local or generated image files can replace those URLs without changing slider behavior.

References

MDN Contributors. (2026a, July 21). *object-fit CSS property*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/object-fit

MDN Contributors. (2026b, April 18). *translateX() CSS function*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/transform-function/translateX